

PRACTICAL 6

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization technique.

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define N 5

sem_t chopstick[N];

pthread_t philosopher[N];

pthread_mutex_t mutex;

void think(int i) {

    printf("Philosopher %d is thinking\n", i);

    sleep(1);

}

void eat(int i) {

    printf("Philosopher %d is eating\n", i);

    sleep(2);

}

void* dine(void* arg) {

    int i = *(int*)arg;

    while (1) {

        think(i);
```

```

    pthread_mutex_lock(&mutex);
    sem_wait(&chopstick[i]);
    sem_wait(&chopstick[(i+1) % N]);
    pthread_mutex_unlock(&mutex);
    eat(i);
    sem_post(&chopstick[i]);
    sem_post(&chopstick[(i+1) % N]);
    printf("Philosopher %d finished eating\n", i);
}
}

int main() {
    int i;
    int phil_num[N];
    pthread_mutex_init(&mutex, NULL);
    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);
    for (i = 0; i < N; i++) {
        phil_num[i] = i;
        pthread_create(&philosopher[i], NULL, dine, &phil_num[i]);
    }
    for (i = 0; i < N; i++)
        pthread_join(philosopher[i], NULL);
    pthread_mutex_destroy(&mutex);
    for (i = 0; i < N; i++)
        sem_destroy(&chopstick[i]);
    return 0;
}

```

```
M ~
ASUS@LAPTOP-LCU90K2P MSYS ~
$ nano dining.c

ASUS@LAPTOP-LCU90K2P MSYS ~
$ gcc dining.c -o dining

ASUS@LAPTOP-LCU90K2P MSYS ~
$ ./dining
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 1 is eating
Philosopher 1 finished eating
Philosopher 2 is eating
Philosopher 1 is thinking
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 3 is eating
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 4 is eating
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 1 is eating
Philosopher 0 is thinking
Philosopher 1 finished eating
Philosopher 2 is eating
Philosopher 1 is thinking
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 3 is eating
Philosopher 4 is eating
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 4 finished eating
Philosopher 0 is eating
```